

System Requirements:

This guide requires at least **12GB Vram or Higher**. I have not tested it with 8GB VRAM configurations.

Tested Hardware:

- RTX 3060 12GB - Training works but is noticeably slower
- RTX 5070 Ti 16GB - Smooth training experience with faster processing times

The performance difference between 12GB and 16GB VRAM is expected and primarily affects training speed rather than capability.

Note1: This guide does not cover LoRA training for Wan 2.2 While it's technically possible using AIToolkit, it demands high VRAM. Since you'll be using **RunPod** for Wan 2.2, it's far better to follow the video tutorial—it's simpler, clearer, and built for that setup.

- **Wan 2.2 Training Guide:** [Click Here](#)

Note2: If you wanna use Ai-toolkit in Runpod you can use my colab notebook: [ClickHere](#)

This article may be a bit lengthy, but rest assured, it is well worth your time. I will also assume that you are already familiar with the process of installing Ai-ToolKit locally.

If you don't know how to install:

- Install the prerequisites: **Git**, **Python 3.11+**, and **Node.js v22+**
- Clone the repo: `git clone https://github.com/ostris/ai-toolkit.git`
- Install Python dependencies: `pip install -r requirements.txt`
- Navigate to the UI folder and install Node dependencies: `cd ui → npm install`
- Start the UI: `npm run dev`

STEP 1: Dataset Preparation

Images: **200+ recommended, 50 minimum** — clear, sharp, varied angles/lighting

Part A — JoyCaption Setup (optimized version by me) for Image Caption

```
git clone https://github.com/official-imvoidid/Joycaption
cd Joycaption
```

Run in order: `GetConda.bat → SetEnv.bat → InstallRequirements.bat → StartTextCaptioner.bat`

1. Bulk Tab, select all images
2. Type/Select what you want the model to tag/focus on
3. Run captioning
4. Download the caption ZIP at the end
5. Unzip it → move all `.txt` files into the **same folder as your images**

Part B — Trigger Word Adder

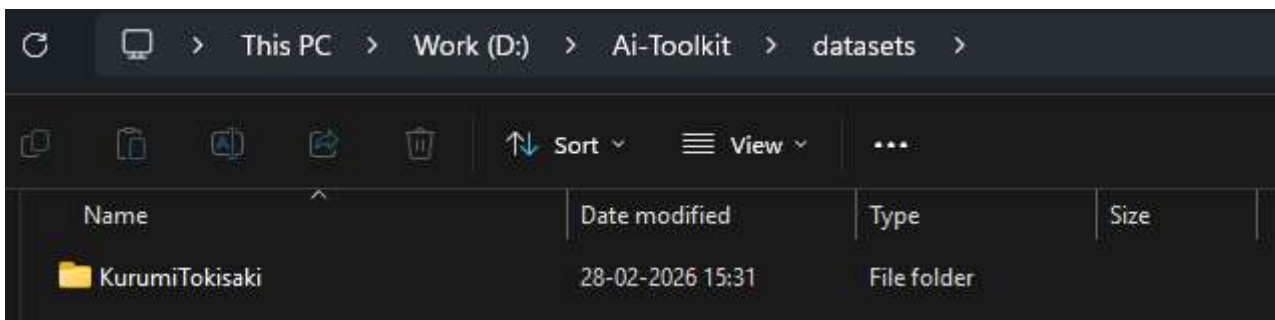
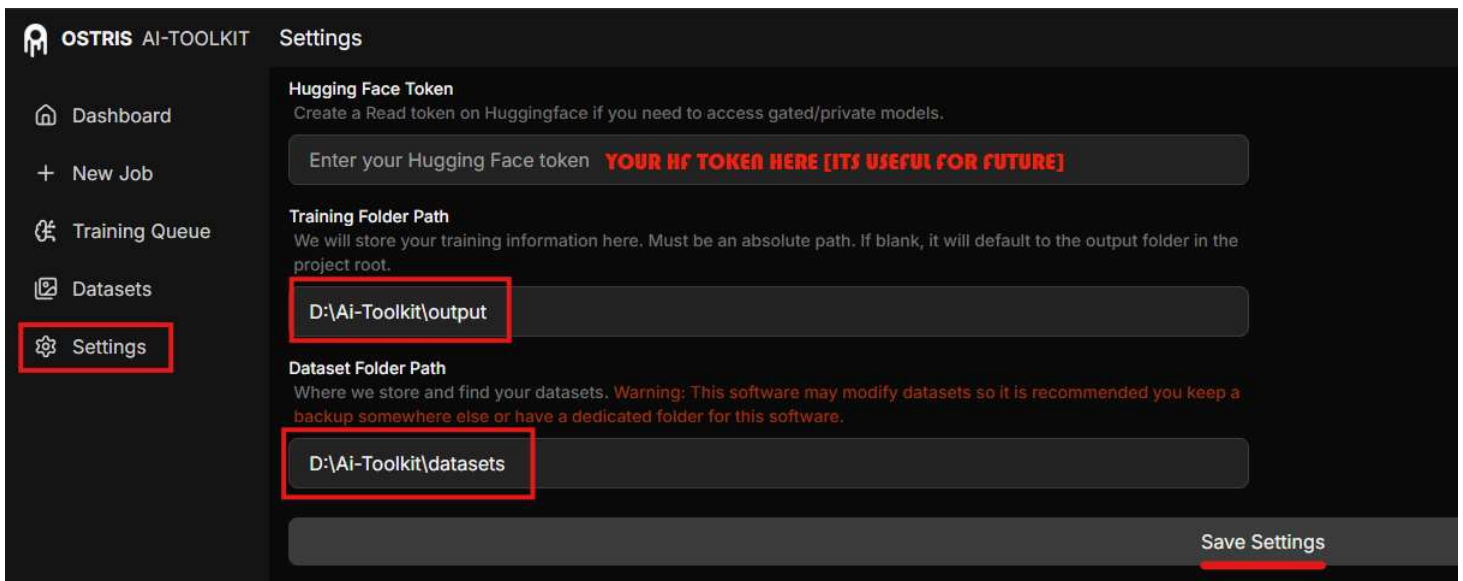
<https://github.com/official-imvoid/Random/blob/main/TriggerWordAdder.py>

No pip install needed — pure Python. Just run `TriggerWordAdder.py` and paste the path of you folder having images and captions/tags ⚠ Critical: Type trigger word as `void,` (with comma+space)

-  `void, other-tags`
-  `voidother-tags`

STEP 2: Pre-Requisite

Make Sure to move this folder containing images & caption to `<Path-To-Your-Ai-Toolkit>\datasets` **And also to apply following settings**



STEP 3: Actual Training

From Here the real training process begins, see these settings [also given as pdf in attachments]

OSTRIS AI-TOOLKIT < New Training Job LoRA Trainer Show Advanced Create Job

JOB

Training Name **lora name**

GPU ID

Trigger Word **SAME TRIGGER WORD AS IN CAPTION**

MODEL

Model Architecture

Name or Path

TARGET **CHANGE**

Target Type

Linear Rank

Conv Rank

SAVE

Data Type

Save Every

Max Step Saves to Keep

TRAINING **see-explanation below**

Batch Size

Gradient Accumulation

Steps

Optimizer

Learning Rate

Weight Decay

Timestep Bias

Loss Type

EMA (Exponential Moving Average) ☐ Use EMA

Text Encoder Optimizations ☐ Unload TE ☐ Cache Text Embeddings

Regularization ☐ Differential Output Preservation ☐ Blank Prompt Preservation

ADVANCED

☒ Do Differential Guidance

Differential Guidance Scale **Does not affect dont change**

DATASETS

Dataset 1 **see-explanation below**

Target Dataset

LoRA Weight

Num Repeats

Default Caption

Caption Dropout Rate

Settings ☐ Cache Latents ☐ Is Regularization ☐ Flipping ☐ Flip X ☐ Flip Y

Resolutions ☐ 256 ☒ 512 ☒ 768 ☒ 1024 ☐ 1280 ☐ 1536

SAMPLE **CHANGE**

Sample Every

Width

Height

Seed

Guidance Scale

Sample Steps

Sample Prompts (1)

Prompt

Width Height Seed LoRA Scale

Parameters [New Job]:

1. JOB Section

- Training Name = Your LoRA name
- Trigger Word = Same word as used in your captions

2. TARGET Section (**CHANGE**) Set as shown: Linear Rank: 40, Conv Rank: 20

3. SAVE Section (*CHANGE*) Data Type: BF16, Save Every: 500, Max Step Saves: 20

4. TRAINING Section

- Batch Size: 1/2 (use 2+ only if VRAM > 16GB)
- Gradient Accumulation: Leave at 1 (no need to touch)
- Steps: Calculate Shown below ↓

$$\text{Total Steps} = (\text{Images} \times \text{Repeats}) \times \text{Epochs}$$

$$\text{Total Steps} = \left(\frac{\text{Number of Images} \times \text{Repeats}}{\text{Batch Size}} \right) \times \text{Epochs}$$

Example:

Images	Repeats	Effective	×10 Epochs	×12 Epochs
200	1	200	2000	2400
50	2	100	1000	1200
50	3	150	1500	1800

Start at 10 epochs → if undertrained → bump to 12, 15, etc.

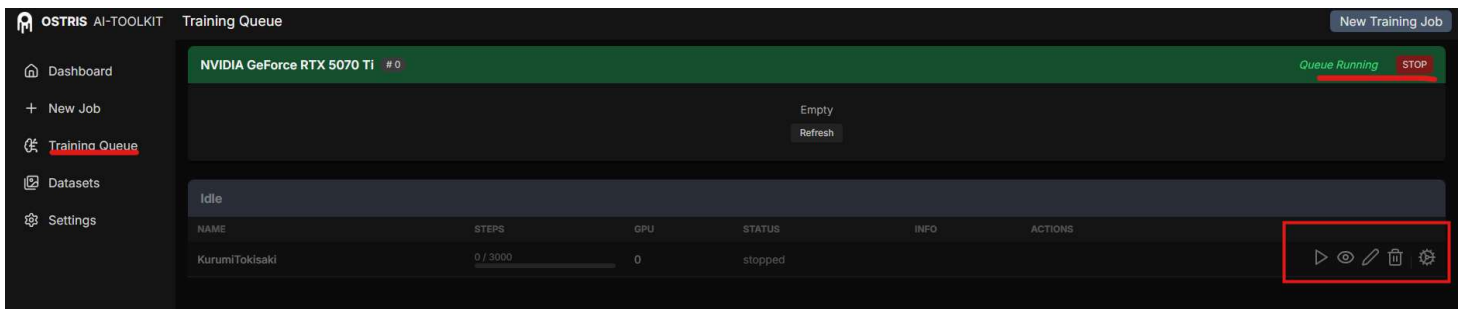
5. DATASET Section

- Click Dataset → select your character's dataset
- Num Repeats:
 - ~50 images → set to 2 or 3
 - 100+ or 200+ images → set to 1

6. SAMPLE Section (*CHANGE*) Fill in a test prompt under Sample Prompts so you can monitor training output visually.

STEP 4: Start Training / Pause / Resume / Edit

Note: If you remove the character output folder from `\Ai-Toolkit\Output`, or delete important files, you will **not** be able to resume or edit training — it will simply restart from 0. You can, however, edit `.safetensors` files directly (e.g., `kurosaki-500.safetensors` or `kurosaki.safetensors`).



Training Locally and Enjoy Lora

-> Q/A

Q1: What tools can I use if I don't have enough VRAM?

A: If your GPU lacks sufficient VRAM, you have two excellent cloud-based alternatives. Use [RunPod](#) for LoRA and Checkpoint training, which provides powerful GPU instances on-demand. For running ComfyUI workflows in the cloud, use [GoogleColab](#) instead. You can get the Jupyter notebook for GoogleColab at my [Github](#). Both options let you train models and generate images without investing in expensive hardware.

Note: Due to recent updates on CivitAI platforms, some file attachments may no longer be visible or accessible. For all resources related to this article — along with additional tools, files, and miscellaneous resources — please refer [HERE](#)